

[↑ Share](#) ...

I accidentally started a process in the ChatGPT Mac app when I ment to do it here. How do I stop/cancel it?

If you started a response and it's still running in the ChatGPT Mac app, there should be a **Stop** button (usually a square icon) in the message area while the response is generating. Click that and it will halt the current generation.

If you accidentally launched an **Agent** task (the newer feature that can work for a while in the background), look for:

- A **Stop**, **Cancel**, or **Interrupt** button in the task pane.
- If that doesn't work, simply **close the chat** or **quit the ChatGPT app** (⌘Q). The task should stop when the session ends. Some Agent tasks may continue briefly until the server receives the cancellation request. [TechRadar](#)

If the app seems stuck:

1. Right-click the ChatGPT icon in the Dock.
2. Choose **Quit**.
3. If it won't quit, press **⌘⇧⌘Q** (Force Quit), select ChatGPT, and click **Force Quit**. [Apple Support Co... +1](#)

Can you tell me what kind of process it is?

- A normal chat response?
- Deep Research?
- Agent mode?
- Something using "Work with Apps"?

The UI is a little different for each one, and I can tell you exactly where the cancel button is.

I was asking it to review some code. I clicked the stop button and then quit the app; hopefully that did the trick.

Yes, that should do it.

For a code review, the process is usually just a normal generation task (even if it was taking a long time because it was reading files). Clicking **Stop** sends a cancellation request, and quitting the app immediately afterward is about as definitive as you can get from the client side.

A few things that may reassure you:

- You won't be charged extra or use additional message quota indefinitely because a stopped response was left running.
- The model isn't sitting there continuing to analyze your code forever after you've quit the app.
- If you reopen the app and go back to that conversation, you'll typically see either a partial response or a message indicating that generation was stopped.

Out of curiosity, was this the same **NextSeason** code review you mentioned earlier, or a different project? If it's NextSeason, we can continue the review here once you've uploaded the zip file.

NextSeason(2).zip

Zip Archive

The same. Here's the file. Please review

I'll inspect the project structure first, then focus the review on the Phase 4 Slice 1 changes and anything that looks risky, inconsistent, or worth cleaning up.

So far the structure looks clean: the Slice 1 work is separated into models, DTOs, service, mapping, view models, and views, with tests added for the core calculation and formatting. I can't run `xcodebuild` in this environment because Xcode's command-line tools are not installed here, so this is a static review rather than a build-verified one.

Thought for 1m 50s >

I reviewed the zip. Overall: **this is good work, and I don't see any obvious architectural red flags**. Slice 1 looks cleanly separated into service, DTOs, domain models, mapping, view models, views, and tests.

I could not run an Xcode build here, but I did inspect the project and code statically.

Main findings

1. The TVMaze API usage looks basically correct.

`/search/shows?q=...`, `/shows/:id`, multiple embeds, `nextepisode`, caching, 429 handling, and User-Agent are all consistent with TVMaze's docs. TVMaze supports `show search`, embedding `nextepisode`, recommends caching expectations, backs off on 429, and recommends a unique User-Agent. 🐙 TVMaze

2. Small mismatch: docs say User-Agent should include contact, code does not.

Docs say:

```
<> Swift
NextSeason/<version> (contact)
```

Code uses:

```
<> Swift
NextSeason/<version>
```

Not urgent, but I'd either update the code later or soften the documentation.

3. Potential date bug in `NextSeasonCalculator` .

This line may mark a season as ended too early on its actual end date:

```
<> Swift
let hasEnded = latest.endDate.map { $0 < now } ?? false
```

Because `endDate` is parsed as midnight UTC, a season ending "today" may count as ended for most of that day. I'd add a test for "season ends today" and probably compare date-only values instead of exact `Date` .

4. `SummaryFormatter` is good enough for MVP, but intentionally limited.

It handles TVMaze's common HTML nicely. Just be aware it is not a general HTML renderer. That's fine for this app.

5. The tests are a strong addition.

The `NextSeasonCalculatorTests` are especially valuable because this is the app's core product logic. I'd add tests for:

```
season end date is today
future undated season while current season is airing
nextepisode with no airdate
empty HTML summary / whitespace-only summary
```

6. Branch state:

In this zip, `Phase4-StepA` and `Phase4-StepB` have already been merged into `main` . That looks fine. I'd delete those branches after you confirm everything builds locally.

Recommendation

I'd keep this Slice 1, make the small date/User-Agent cleanup, run the full test suite in Xcode, then move on to Slice 2. This is a solid foundation.

NextSeason(3).zip
Zip Archive

Here it is again with the changes made. Please review again and let me know if you see anything else to address.

I'll compare this version against the previous review points first, then scan for any new issues introduced by the changes.

Thought for 31s >

I reviewed it again. The prior issues look addressed.

Only things I'd consider, none urgent:

1. **Branch state:** this zip is on `Phase-4-Review-Items`, not `main`. Merge it when tests pass.
2. **User-Agent:** docs now match code. Good.
3. **Date-only bug:** fixed cleanly with `TVMazeDate.isBefore/isAfter/isOnOrBefore`, and the new "end date is today" test covers it.
4. **One tiny future consideration:** `NextSeasonCalculator` still treats an undated `nextEpisode` in a new season as `.airing`. That's reasonable, but slightly optimistic. Fine for MVP.

I don't see anything else I'd block on. I'd run tests in Xcode, merge this review branch, and move on to Slice 2.